

Raku++ |

A from-scratch Raku, hand-written in C++17.

Interpreter + native compiler

runs in the **browser** via WebAssembly

installs modules, **zef-compatible**

100.00% of Roast's tests

v5.0.0

— A SECOND IMPLEMENTATION, WITH RAKUDO AS ITS REFERENCE

One implementation of the **language**, from lexer to native binary.

A hand-written lexer, parser, and tree-walking evaluator for real **Raku** — classes, roles, grammars, regexes, multi-dispatch, junctions, lazy sequences, a bignum tower, Unicode-correct strings, concurrency. It can also **compile** a program to a standalone native executable, and — as **Raku.js** — run client-side in a browser.

- **C++17, no deps**

Hand-rolled bignum, Unicode tables, and regex engine. Builds anywhere CMake and a C++17 compiler run.

- **Interpret *or* compile**

The same binary tree-walks a script or transpiles it to C++ and compiles native.

- **Measured, not claimed**

Graded test by test against **Roast**, the specification suite the Raku community wrote alongside Rakudo — and checked behaviour by behaviour against Rakudo, the reference compiler.

— GRADED AGAINST ROAST · MEASURED ON V5.0.0 · ROAST COMMIT 1F521D798

Every test Roast expects to **pass**.

100.00%

218,421 / 218,422 tests pass across the 1,424 files of Roast's `spectest.data`, counting only the tests Roast expects to pass — nothing skipped, nothing marked todo.

1,423 / 1,424

files pass **all or nothing**. The one test left reads `.eof` on a terminal; Roast marks it todo on macOS by release name, and the machine of record runs a release newer than the list.

767

files moved to fully passing since v4.0.1, with **0 regressed** — 249 commits, synopsis by synopsis, then the long tail one file at a time.

None of this is measurable without **Roast** — years of the Raku community writing down what the language means, test by test. Every release is gated on it with **zero fully-passing-file regressions**, and on **performance** too; the full list runs in about 40 seconds through a harness written in Raku and run by Raku++ itself.

Big-language Raku, most corners lit.

NUMBERS

Exact by default

Arbitrary-precision `Int`,
exact `Rat` — `0.1+0.2-0.3`
is exactly `0` — `Num`,
`Complex`, allomorphs.

OBJECTS

class · role · grammar

Multi-dispatch,
`BUILD` / `TWEAK`, mixins,
`augment`, full `.^`
introspection, `.wrap`.

REGEX

Grammars that parse

Backtracking engine, Unicode
classes, `token` / `rule`,
`.parse` with action classes &
`make`.

SIGNATURES

multi & proto

Type / literal / `where`
constraints, destructuring,
coercion types, `callsame`.

FUNCTIONAL

Lazy & higher-order

Closures, `*` `WhateverCode`,
junctions, `gather` / `take`,
feeds, `1,2,*,*...Inf`.

METAPROG

Your own operators

User ops in all six categories
with precedence traits and
meta-operators over them;
`RakuAST` (`.AST`,
`.DEPARSE`, `.EVAL`) and
`slangs`.

CONCURRENCY

Real OS threads

`start` / `await`,
`Supply` / `react` / `whenever`,
`Channel`, atomics, `hyper`
`for` over worker threads; true
parallelism **on by default**.

UNICODE

Grapheme-correct

UAX #29 (emoji ZWJ),
NFC/NFD/NFKC/NFKD, UCA
collation — generated
UCD/UCA 17.0 tables.

— ONE SOURCE, FIVE TARGETS

Four ways to run — and a fifth in the browser.

`rakupp prog.raku`

Interpret — the default; covers the whole language.

`--bundle -o prog`

Standalone binary that re-parses the source at startup.

`--aot -o prog`

Standalone binary embedding the already-parsed AST.

`--exe -o prog`

Transpile to C++ and compile native — anything unsupported falls back to `--bundle` automatically.

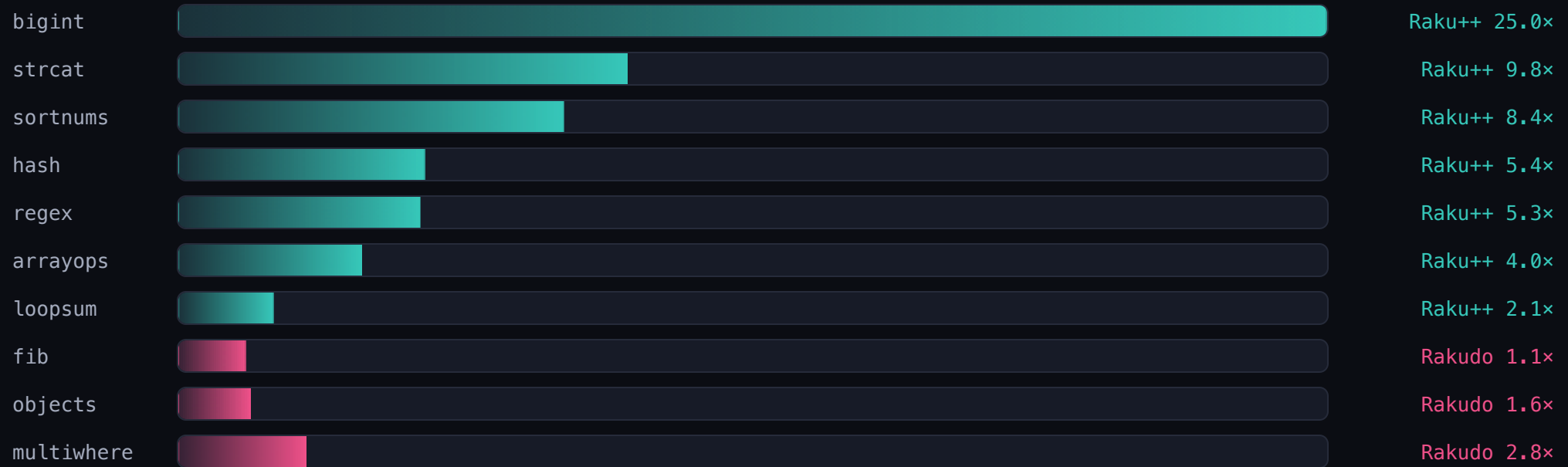
`Raku.js → WASM`

In the browser — same semantics as native, client-side, no server, with an embeddable playground.

Opt-in, off by default: `--precomp-modules=on` caches the parsed form of everything a program uses. New in v5.0.0 and marked work in progress: `--jit` and `--cnp` compile hot loops *while the program runs* — the second by copy-and-patch, with no C++ compiler needed.

— INTERPRETER · TIME RELATIVE TO RAKUDO V2026.08 · BOTH NATIVE ARM64, ONE M3, BYTE-IDENTICAL OUTPUT

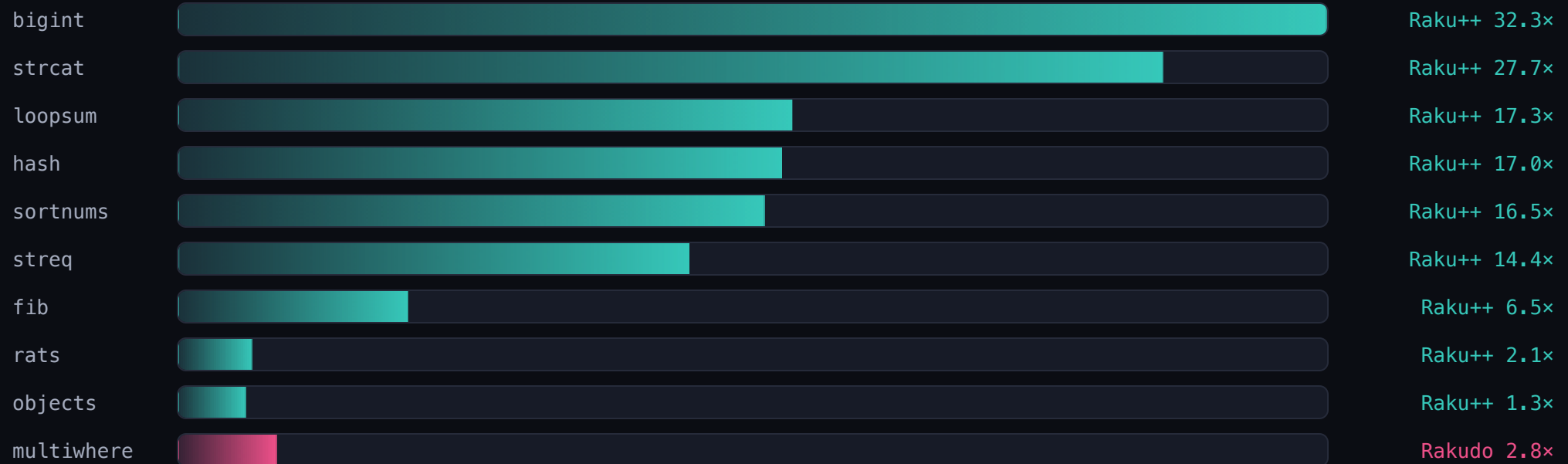
Startup and throughput, **measured.**



3.7 ms to start interpreting and 3.0 ms compiled, against Rakudo's 81.6 ms – which loads a full VM that then specialises code as it runs, and that is where its leads come from. Interpreted, Raku++ is ahead on thirteen of seventeen kernels; Rakudo leads on fib and streq (1.1×), objects (1.6×) and multiwhere (2.8×). v5.0.0 also interprets 5–40% slower than v4.0.1: the price of the Roast campaign, and where v6 starts.

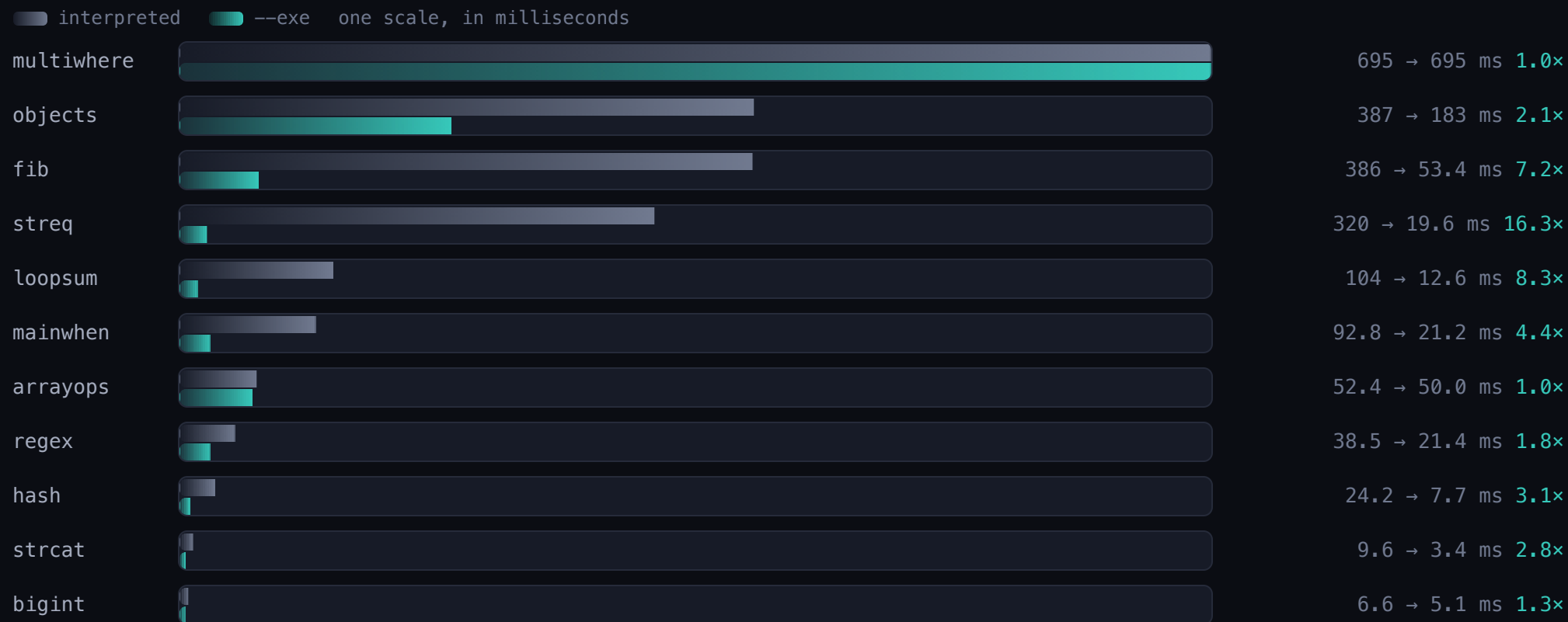
— RAKUPP --EXE · TRANSPILED TO C++, COMPILED NATIVE · TIME RELATIVE TO RAKUDO V2026.08

Compiled, the tree-walk goes away.



Sixteen of seventeen kernels run **1.3×** to **32.3×** faster than Rakudo once compiled. The closest is objects – class, has, method dispatch – where MoarVM's spesh does the work a native binary gets from its compiler. multiwhere is not compiled at all yet: codegen declines a where on a multi candidate and bundles the interpreter, so Rakudo leads it by **2.8×**.

Interpret or compile: what compiling buys.



Compiling pays most where a tree-walker re-dispatches a tiny body many times – streq, loopsum, fib – and almost nothing where the time is spent inside a runtime method both modes share: arrayops (.grep/.map/.sum) and bigint (the BigInt multiply). multiwhere is the fallback at work: codegen declines it, so the binary bundles the interpreter. Startup is 3.7 ms interpreted and 3.0 ms compiled; the interpreter covers the whole language, and --exe compiled 773 of the 1,189 fully passing Roast files natively when last counted, bundling the rest.

— EVERY DISTRIBUTION IN THE ECOSYSTEM, SWEEP FRESH ON THE V5.0.0 RELEASE BINARY

The whole ecosystem, **as the test.**

1,019 / 2,547

distributions **install and pass their own test suites** — the files their authors wrote, not an API probe — from a store seeded only with the 478 most depended-on modules. The previous fresh sweep gave 860.

216

never reached their own tests: a **dependency failed first**. Fixing one root distribution is the fastest way to move many.

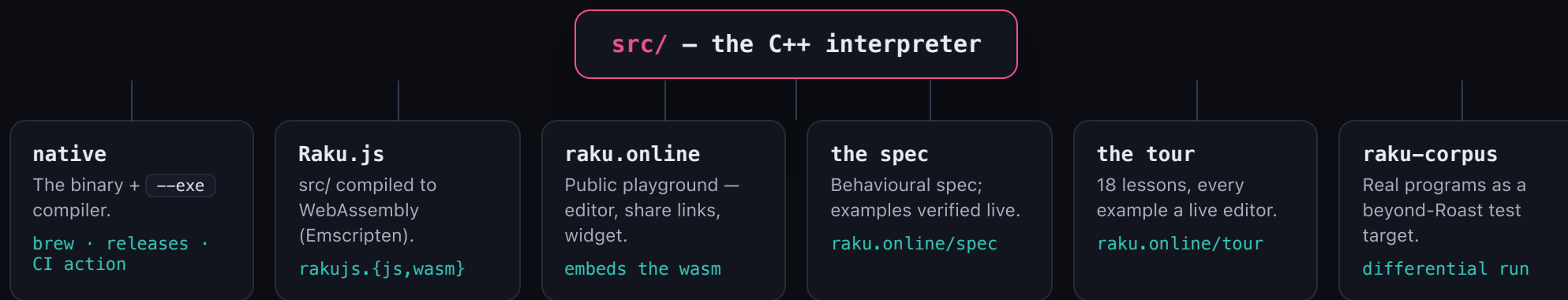
1,791

is what **Rakudo** passed of the 2,529 in the previous index, on the same machine through the same harness — the ceiling to read the figure against. The rest need libraries or a network the box does not have.

The sweep is not a scoreboard — it is **the worklist**. What fails decides what gets built next, and nothing is special-cased for a module: the fixes land in the parser, the runtime and dispatch, which is why chasing one distribution keeps paying out in others. Every distribution, with how it ran, is browsable at `raku.online/modules/ecosystem`.

— ONE INTERPRETER BEHIND EVERY SURFACE

A small constellation, one source of truth.



Everything downstream ships the *same* interpreter — the `wasm`, the sub-sites, the Homebrew tap, and the release binaries all trace back to `src/`. Neither the spec nor the tour hosts an engine of its own, so both inherit a new one for free the moment the playground redeploys.

— MID-SIZE PROGRAMS, EACH LEANING ON A DIFFERENT PART OF THE LANGUAGE

Real programs, written **in** Raku.

Scheme

A Lisp interpreter on a Raku grammar.

GRAMMAR · EVAL

Forth

Stack machine, also grammar-parsed.

GRAMMAR

Markdown → HTML

Converter — also runs in the browser.

REGEX · WASM

JSON

Parser + formatter, browser-ready too.

GRAMMAR · WASM

Pastebin

HTTP service on raw sockets.

SOCKETS

Chat server

Concurrent, many-client.

THREADS

Key-value store

Its own wire protocol.

SOCKETS

rakus

A static-file HTTP server.

SOCKETS

Plus three interpreters for *other* languages, each written in Raku: **JavaScript/TypeScript**, **Perl 5** (Raku parsing its own ancestor), and **Python 3** — off-side rule included, via a CPython-style INDENT/DEDENT tokenizer — with examples byte-identical to `node`, `perl` and `python3`. Every one of the 24 bundled `examples/` also compiles natively with `--exe`, byte-for-byte identical.

Modules from the ecosystem, **as installed.**

Raku++ installs modules with **its own installer** — `rakupp install JSON::Fast`, a checkout, or a URL — into the **same store zef uses**. A distribution installed by either tool is found by `use` under either engine.

THE BAR

48 / 59 distributions

in the per-release battery pass *their own* test suites — the files zef runs at install time, not a one-line API probe. It is the per-release QA instrument, not the ecosystem picture — the sweep is that.

THE SET

Top 50 by reverse-deps

JSON::Fast, XML, YAMLish, URI, File::Temp, File::Find, Terminal::ANSIColor, Digest, Encode — ranked over the ecosystem archive, version-pinned.

THE METHOD

Every fix is general

No module accommodations: `is raw` write-back, `CALL-ME` by name, `method FALLBACK`, `END` at process exit — all landed as interpreter fixes.

Far end of the same road: a live `HTTP/1.1 200 OK` over TLS — Cro's client on `I0::Socket::Async::SSL`, talking to the system OpenSSL through Raku++'s own NativeCall. Getting there was a chain of ~13 bugs, and nearly every one was a general correctness bug.

— BEYOND WHAT ROAST'S UNIT GRANULARITY EXERCISES

Real applications run **unmodified**.

RAKU-COURSE

~1,500 pages, byte-identical

The full course static-site generator regenerates the entire site byte-for-byte identically to Rakudo — including the real zef-installed

`YAMLish` module.

COVID.OBSERVER

End-to-end build

The site builder runs start to finish on Raku++ — a genuine data-processing application, not a curated snippet.

```
# exact arithmetic, lazy infinite sequence, grapheme strings –
# the things that make real programs match, not just tests
say 0.1 + 0.2 - 0.3;           # 0 (exact Rat)
say (1, 2, *+* ... *)[^10];    # (1 2 3 5 8 13 21 34 55 89)
say "café".chars;              # 4 (graphemes, not bytes)
```

More than a way to **run a file.**

V3.28.0

RakuAST, end to end

`.AST`, `.DEPARSE`, `.EVAL`, `visit-children`, `.rakudoc` — built as a **view** over the existing parse, not a second front end, so the ordinary path pays nothing for it: worst +2.7%, mean **-1.3%** over sixteen kernels.

V3.28.0

--fmt

A formatter with gofmt's promise — zero configuration, run it on every save. Whitespace only, and **three gates before a byte is written**: the input must parse, the result must be the same program, and formatting it again must change nothing.

V3.28.0

use L10N::DE;

...and the rest of the file is German. **Eleven languages**, off the same table RakuAST already reads. Upstream this is a slang; here it is a rewrite of the token stream, which is why it needs no flag when Rakudo wants `RAKUDO_RAKUAST=1`.

TARGETS

--target=js

Transpile a program to JavaScript for Node, Bun, Deno or a browser — with `--verify` to check the result against the interpreter that produced it.

PROTOCOLS

MCP · Jupyter · LSP

The interpreter over the **Model Context Protocol**, so an AI agent gets a Raku session with exact arithmetic and grammars as structured extraction. A **Jupyter kernel** that speaks ZMQ and signs its own messages: no ZeroMQ, no Python module. And `--lsp`.

EMBEDDING

Six host languages

`librakupp` behind a small C ABI, wrapped for **Python, JavaScript, Go, Rust, C++ and Wolfram**. Each gets the same two things: run Raku, and parse with Raku grammars where the grammar stays a `.raku` file.

The C API and its bindings shipped in **v4.0.0**, with Raku grammars callable from every one of those host languages.

What comes after 100%.

- ▶ **Code Rakudo runs that Raku++ refuses.** Parse-time checks added while chasing Roast files now reject some ecosystem code; the battery compile scan found 25 module files in 12 distributions that compiled before, and the target is none.
- ▶ **Crashes and silent wrong answers** — a data race in `evalCall` under `start`, and `--target=js` programs that disagree with the interpreter, 102 of them when the plan was written.
- ▶ **The workloads where Rakudo leads.** At ten times the work, `objects` reads 2.9× and `multiwhere` 4.8× — `spesh` doing what `spesh` is for. `v6`'s first number is every `perf-guard` kernel at or below Rakudo at steady state, and `v5.0.0`'s own interpreter slowdown is the first thing it pays back.
- ▶ **Memory per value**, and **compile and load time halved** — an `--exe hello`'s translation unit takes 0.93 s today.
- ▶ **`--exe` without falling back** on $\geq 90\%$ of the fully passing Roast files; 65% compiled natively when it was last counted.
- ▶ **use in the browser** — modules in the playground, in embeds and under `--target=js`.

Each of those is a number a stranger can re-measure, written down before the code — the same way the Roast figure was.

— WHAT A SECOND IMPLEMENTATION STANDS ON

Built on the work of the **Raku** community.

ROAST

The specification

Every figure on slide 3 exists because Roast does: a test suite that says what Raku means precisely enough for a second engine to be graded against it.

RAKUDO

The reference compiler

When Roast is silent, Rakudo answers. Eighteen sheets of its behaviour, each item with a probe; 1,006 documentation examples byte-identical on both; the module battery judged against each distribution's own run under Rakudo.

THE ECOSYSTEM

Other people's code

2,547 distributions whose authors wrote tests — which turns the whole ecosystem into a test suite — and the documentation at docs.raku.org.

Raku++ shares no code with Rakudo, and it would not exist without it. Thank you to everyone who built the language, its reference compiler, its tests and its modules.

— MACOS · LINUX · WINDOWS · THE BROWSER

Try **Raku++** today.

```
$ curl -fsSL https://raku.online/install.sh | sh
```

```
$ rakupp upgrade # later, in place
```

```
➤ raku.online # or run it in the browser, no install at all
```

github.com/ash/rakuppraku.onlineraku.online/specraku.online/tour

Neither installer needs root. On Windows, `irm ../install-windows.ps1 | iex` or a setup wizard with an Add/Remove Programs entry; on macOS, a Homebrew tap as well. Prebuilt archives on every GitHub Release — macOS universal, Linux x86_64 and ARM64, Windows x64, OpenBSD — plus the WebAssembly bundle. No runtime dependencies, anywhere.